

DEVELOPMENT

BobaLink — Development Guide

Revision: 1 **Last modified:** 2026-06-10T21:00:00Z **Scope:** Architecture, layout, testing, and contribution rules for BobaLink (extension/).

Authority: extension/package.json, extension/wxt.config.ts, docs/browser_extension/IMPLEMENTATION_PLAN.md, docs/browser_extension/Status.md (Rev 4), project CLAUDE.md + constitution/.

Accuracy note (§11.4.6): the architecture and commands below reflect the real tree at HEAD 2011810. Phases still open are marked **(in progress)**.

Stack

- **WXT** (wxt ^0.19.0) — the WebExtension build framework; generates the MV3 manifest from wxt.config.ts and bundles entrypoints into .output/.
 - **TypeScript** (^5.7.0, strict) — npm run compile is tsc --noEmit.
 - **Vitest** (^2.1.8) + @vitest/coverage-v8 — unit/integration/etc. tests.
 - **Playwright** (@playwright/test ^1.49.0) — end-to-end (real MV3 extension load).
 - **ESLint** (^9.17.0) + Prettier — lint/format.
 - Runtime dependency: webextension-polyfill.
 - Node ≥ 20, ESM ("type": "module").
-

Architecture

The pipeline, end to end:

page DOM

→ parser (magnet / bencode / .torrent → SHA-1 infohash)

- scanner (link-scanner + text-scanner, MutationObserver, site selectors)
- orchestrator (cross-scanner dedup by infohash)
- background (service-worker message router; context menus; commands; alarms)
- boba-client (POST to the Boba merge service on :7187; token header; retry)
- queue (offline FIFO; persists across SW restart; auth-injected send)

Module map (under extension/src/):

- **parsers** — magnet URI parsing, bencode decoding, .torrent → SHA-1 infohash (SHA-1(bencode(info)) via WebCrypto). Dedup key is the lowercase infohash.
- **scanners** — link-scanner / text-scanner + content/scanner; stable infohash-based IDs; Shadow-DOM/iframe aware; 500 ms MutationObserver debounce.
- **orchestrator** — runs the scanners and deduplicates across them.
- **background/index.ts** — the service-worker message router: dispatches context-menu clicks (boba-link-send / boba-link-send-all / boba-link-send-group), keyboard commands (send-to-boba / scan-page / open-dashboard), and chrome.alarms keep-alive + periodic health checks.
- **api/boba-client.ts** — the Boba merge-service client (Phase 4, **in progress**): posts magnets/URLs to :7187, presents the optional decrypted Boba API token as an Authorization/X-Boba-Token header, with retry/backoff and an AbortController timeout.
- **api/queue.ts** — the offline FIFO queue; enqueues on send failure and flushes when the backend recovers; durable across service-worker restart.
- **tabgroups/index.ts** — dedupe + batch dispatch across a Chrome tab group, invoked from the background MENU_SEND_GROUP handler.
- **shared/** — constants.ts, crypto.ts (AES-GCM-256 / PBKDF2), errors.ts, events.ts, logger.ts, storage.ts, utils.ts.
- **options/options.ts** + **popup/** — the 7-tab options page and the detected-list popup with Send / Send-All.

WXT entrypoints (thin wrappers)

WXT discovers entrypoints under extension/src/entrypoints/:

```
src/entrypoints/
  background.ts    → background.js service worker
  content.ts       → content-scripts/content.js
```

```
popup/index.html      → popup.html
options/index.html    → options.html
```

These are **thin wrappers** over the logic modules listed above — the heavy logic lives in `src/`

`{parsers,scanners,background,api,options,popup,shared,...}`/, so the existing unit tests import the logic modules directly and stay valid regardless of WXT's entrypoint plumbing (`Status.md` Rev 4, “WXT build wiring — COMPLETE”).

`wxt.config.ts` sets `srcDir: "src"`, `outDir: ".output"`, the MV3 manifest (least-privilege permissions, localhost-only `host_permissions`, the CSP, the three keyboard commands), and `@wxt-dev/auto-icons` to rasterize icon sizes.

Running tests

From `extension/`:

```
npx vitest run          # the full unit/integration/security/perf/
                        etc. suite
npm run test            # === vitest run --coverage
npm run test:watch      # vitest in watch mode
npm run test:live       # vitest run --config tests/live/
                        vitest.live.config.ts
npm run test:e2e        # playwright test (real MV3 extension
                        load)
```

- The Vitest corpus lives under `extension/tests/` `{unit,perf,stress,chaos,integration,security,e2e}` and alongside source in `src/**/*. As of Status.md Rev 4 a same-session npx vitest run reported Tests 413 passed (413) across 37 spec files.`
- **test:live** targets a live backend; live integration uses a `require_backend(7187)` guard that **SKIPS with reason** when `:7187` is down (never fail-open, §11.4.3/§11.4.68).
- **test:e2e** loads the real built extension via Playwright (`launchPersistentContext + --load-extension`, real id from the SW target). The MV3-load e2e is **operator-gated SKIP** in a headless sandbox where extension load is unsupported.
- Anti-bluff is the gate: each test asserts a user-observable outcome and must fail against a no-op stub (`IMPLEMENTATION_PLAN.md` §0.2). Challenges under `challenges/extension/` and `challenges/security/` drive the real orchestrator/client/audit end-to-end and are mutation-verified.

Type + lint, run by `ci-ext.sh` and individually:

```
npm run compile      # tsc --noEmit
npm run lint         # eslint src/ tests/ --ext .ts
npm run format:check # prettier --check "src/**/*.ts"
```

Constitution constraints (read before contributing)

BobaLink inherits the Boba `CLAUDE.md` and the `constitution/` submodule. The load-bearing rules for this subproject:

- **Anti-bluff (§11.4).** Every test/Challenge must prove the feature works for the end user — assert user-observable outcomes (DB rows, response bodies, DOM text), and each test must FAIL against a no-op stub. The reference suite's 10 known bluffs are remediated, not adopted.
- **TDD (§11.4.43/§11.4.115).** Write the failing RED test first (reproducing the real defect / driving the real feature), watch it fail, then write minimal code to GREEN. “Test added after the fix” is a bluff.
- **NO CI/CD — ever (Hard Stop §1).** No `.github/workflows/`, no pipelines, no git hooks. Validation is the manual `extension/ci-ext.sh`.
- **No force-push (§11.4.113); fetch-first (§11.4.71); SSH remotes only.**
- **§11.4.10 credentials.** Delegate-by-default; no embedded/decryptable secret by default; no fixed/empty-passphrase decrypt; no real credential in any test/fixture/doc.
- **100% multi-type coverage (§11.4.27/§11.4.85).** unit/integration/e2e/security/ load/scaling/chaos/stress/performance/benchmark/UI/UX + Challenges + HelixQA.
- **Mandatory code review (§11.4.125/§11.4.134/§11.4.142)** of every change, iterate-until-GO, before it is “complete”.

The single real backend is the **Boba merge service on :7187** — never qBittorrent directly, never port 8080 (the reference :8080 values were all retargeted to :7187; see `src/shared/constants.ts`).

Where the docs / Status / ledger live

All under docs/browser_extension/:

- **Status.md** (+ .html/.pdf) — the authoritative per-phase status (Rev 4 is the real state).
- **Status_Summary.md** (+ siblings) — the two-audience summary companion.
- **IMPLEMENTATION_PLAN.md** (+ siblings) — the 9-phase master plan with the hard constraints, backend additions (BE-1/2/3), and adopt/refactor/rewrite map.
- **coverage_ledger.md** (+ siblings) — the feature × test-type coverage ledger.
- **USER_GUIDE.md**, **INSTALL.md**, **DEVELOPMENT.md** (this file) — the Phase-9 user/install/dev docs (+ .html/.pdf siblings per §11.4.65).
- Deeper provenance: docs/browser_extension/_analysis/ and _plan/.

The script companion for the gate is docs/scripts/ci-ext.md (§11.4.18).

HelixQA banks: submodules/helixqa/banks/boba-bobalink.yaml

(symlinked into challenges/helixqa-banks/).